Relational Databases and Joins

Preamble

Packages and Datasets

This lab will primarily use the `dplyr` package.

```
# load the following packages
library(dplyr)
```

The lab's examples will use two *databases*:

```
orders = read.csv("https://raw.githubusercontent.com/ds4stats/r-tutorials/master/merging/data/orders.csv
customers = read.csv("https://raw.githubusercontent.com/ds4stats/r-tutorials/master/merging/data/custome
```

- **Description**: A contrived example of a customer database that is small enough to visually inspect.

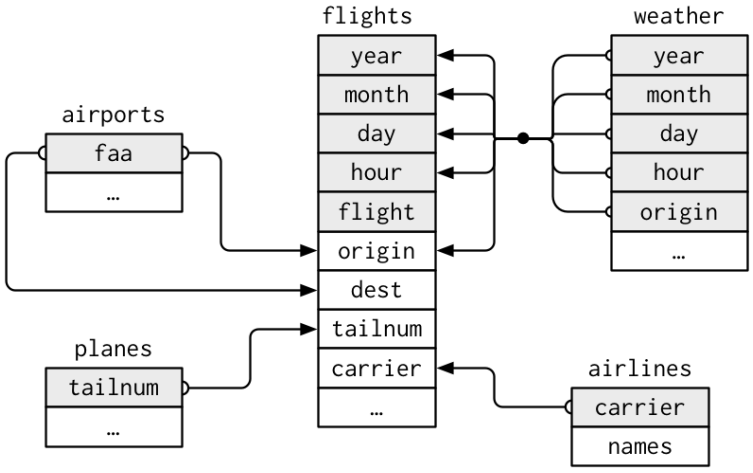
```
library(nycflights13)
data("flights")
data("planes")
data("weather")
data("airports")
```

- **Description**: Data originating from the US Bureau of Transportation pertaining to all flights that departed from New York City in 2013.

Databases

So far we've only dealt with data that could be stored in a single spreadsheet; however, many real world applications require more flexibility.

Data are often stored in *databases*, or structured collections of records. Essential to a database is its *relational structure*. For example, here is the relational structure of the `nycflights13` database:



From this diagram you should notice a few things:

- Information about a plane could be added to `flights` via the "tailnum" variable
  - This variable exists in both data files
- Information about the airport that a plane is departing from or arriving at can be added to `flights` using the "faa" variable in `airports`
  - The corresponding *key* in `flights` is either "origin" or "dest"

Keys

Matching data across the different components of a database is done using *keys*:

- A **primary key** is a variable (or set of variables) that uniquely identifies an observation in its own table
  - "tailnum" is a primary key in `planes` as no tail number appears more than once in this data frame

```
planes |> count(tailnum) |> filter(n > 1)  ## Primary key (unique to every plane)
```

```
## # A tibble: 0 x 2
## # i 2 variables: tailnum <chr>, n <int>
```

- "faa" is a primary key in `airports` as no entry appears more than once in this data frame

```
airports |> count(faa) |> filter(n > 1) ## Primary key (unique to each airport)
```

```
## # A tibble: 0 x 2
## # i 2 variables: faa <chr>, n <int>
```

- A **foreign key** is a variable (or group of variables) in *another table* that can be linked to a primary key.
  - "tailnum" is a foreign key in the `flights` data file, as it can be linked to "tailnum" in the `planes` data file
  - Notice that "tailnum" is *not* a primary key in the `flights` file (as many values appear more than once)

```
flights |> count(tailnum) |> filter(n > 1)  ## Foreign key (unique to planes, but not flights)
```

```
## # A tibble: 3,873 x 2
##    tailnum     n
##    <chr>   <int>
##  1 D942DN      4
##  2 N0EGMQ    371
##  3 N10156    153
##  4 N102UW     48
##  5 N103US     46
##  6 N104UW     47
##  7 N10575    289
##  8 N105UW     45
##  9 N107US     41
## 10 N108UW     60
## # i 3,863 more rows
```

A primary key and a foreign key combine to form a **relation**. Relations are used to link information in one data file to another.

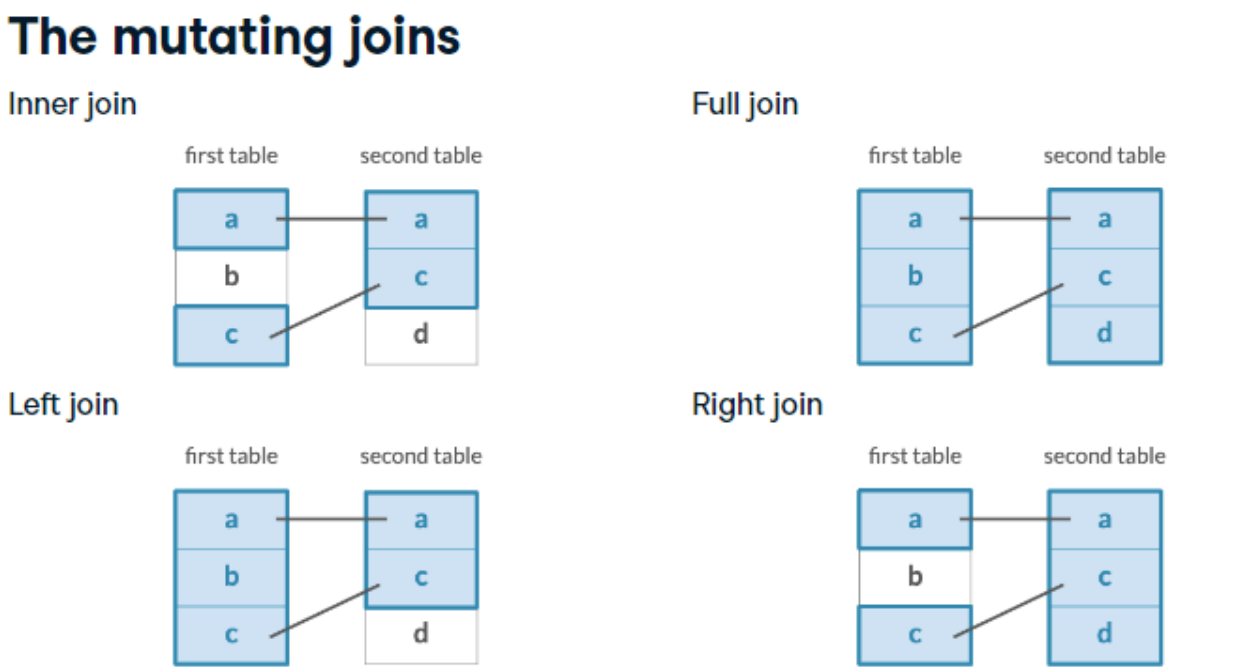
Mutating Joins

The goal of a *mutating join* is to combine variables from two different data frames, "X" and "Y".

There are three important types of mutating joins:

- **Left Outer Join**: Keep *all* observations in "X" and add information from any matching records in "Y", filling with `NA` for records in "X" that do not have a match in "Y".
- **Full Outer Join**: Keep *all* observations in *both* "X" and "Y", filling with `NA` for records in "X" without a match in "Y" *and* for records in "Y" without a match in "X"
- **Inner Join**: Keep *only* observations with records in *both* "X" and "Y", dropping any records that aren't present in both data frames.

In most circumstances you can expect to use a left outer join, as this approach attempts to preserve all of the records in "X" while attaching additional variables from "Y". The different mutating joins are summarized in the graphic below:



Blue sections illustrate observations (rows) that will be present in the combined data frame, while the white sections represent observations that will be omitted. We will not discuss Right Joins, as they are equivalent to a Left Outer Join with the roles of "X" and "Y" reversed.

Lab

Question #1: Use the following data tables in the `nycflights13` database:

```
data(flights)
data(airlines)
```

- **Part A**: Is the variable "flight" a primary key in the `flights`? Why or why not?
- **Part B**: What keys form the relation needed to add the full airline name (contained in `airlines`) to the records in `flights`?
- **Part C**: Is the information in these data tables (excluding `airlines`) wide or long form?
- **Part D**: What are some advantages of storing information in wide form? What about advantages of long form?
- **Part E**: Rather than storing all the information in a single large data frame, relational databases often store information across several smaller, related tables. Why might this be advantageous? What potential problems could arise if all the information were stored in one large table?

Left Joins

Consider the data frames "orders" and "customers":

```
print(orders)
```

```
##   order id   date
## 1     1  4 Jan-01
## 2     2  8 Feb-01
## 3     3 42 Apr-15
## 4     4 50 Apr-17
```

```
print(customers)
```

```
##   id    name
## 1  4   Tukey
## 2  8 Wickham
## 3 15   Mason
## 4 16  Jordan
## 5 23   Patil
## 6 42     Cox
```

Notice that the variable "id" is a primary key (unique identifier) in each data frame.

The `left_join()` function is used to perform a left outer join, adding the customer information to each order using the column "id" as the join's key:

```
left_join(x = orders, y = customers, by = "id")
```

```
##   order id   date    name
## 1     1  4 Jan-01   Tukey
## 2     2  8 Feb-01 Wickham
## 3     3 42 Apr-15     Cox
## 4     4 50 Apr-17    <NA>
```

Because no entries in the `customers` data frame had an "id" of 50, a missing value, `NA`, appears as the name for that order in the combined data frame.

In most situations, left joins *are not symmetric*, so your choice of "x" and "y" matters:

```
left_join(x = customers, y = orders, by = "id")
```

```
##   id    name order   date
## 1  4   Tukey     1 Jan-01
## 2  8 Wickham     2 Feb-01
## 3 15   Mason    NA   <NA>
## 4 16  Jordan    NA   <NA>
## 5 23   Patil    NA   <NA>
## 6 42     Cox     3 Apr-15
```

As seen above, customers with "id" numbers of 15, 16, and 23 did not have any matching records in the `orders` data frame, resulting in missing values for "order" and "date" in the combined data frame.

Question #2: Consider the example join demonstrated below. Given our initial description of a left outer join, you'd expect the left join using `table2` as the "x" argument in the code below to produce a data frame with 3 rows (one for each observation in `table2`). Instead, in this scenario, it produces a data frame with 6 rows. Why do you think this happens?

```r
table1 = data.frame(key = c("a", "a", "b", "b", "b", "c"), X = 1:6)
table2 = data.frame(key = c("a", "b", "c"), Y = c("X", "Y", "Z"))

left_join(x = table2, y = table1, by = "key")
```

```
##   key Y X
## 1   a X 1
## 2   a X 2
## 3   b Y 3
## 4   b Y 4
## 5   b Y 5
## 6   c Z 6
```

Full Outer Joins

The `full_join()` function will perform a full outer join of two data frames:

```r
full_join(x = orders, y = customers, by = "id")
```

```
##   order id   date    name
## 1     1  4 Jan-01   Tukey
## 2     2  8 Feb-01 Wickham
## 3     3 42 Apr-15     Cox
## 4     4 50 Apr-17    <NA>
## 5    NA 15   <NA>   Mason
## 6    NA 16   <NA>  Jordan
## 7    NA 23   <NA>   Patil
```

Notice how the order without a customer (id 50) as well as the three customers without orders (ids 15, 16, and 23) are *all included* in the resulting data frame alongside the three records that had matches (ids 4, 8, and 42).

Full outer joins *are symmetric* in terms of which records are included. Reversing "x" and "y" does not change which records are matched, although the resulting data frame may be organized differently:

```r
full_join(x = customers, y = orders, by = "id")
```

```
##   id    name order   date
## 1  4   Tukey     1 Jan-01
## 2  8 Wickham     2 Feb-01
## 3 15   Mason    NA   <NA>
## 4 16  Jordan    NA   <NA>
## 5 23   Patil    NA   <NA>
## 6 42     Cox     3 Apr-15
## 7 50    <NA>     4 Apr-17
```

Inner Joins

The `inner_join()` function is used to perform an inner join on two data frames:

```r
inner_join(x = orders, y = customers, by = "id")
```

```
##   order id   date    name
## 1     1  4 Jan-01   Tukey
```

```
## 2     2  8 Feb-01 Wickham
## 3     3 42 Apr-15     Cox
```

Inner joins are also *symmetric* in terms of which records are included. Reversing "x" and "y" does not change which records are matched, although the resulting data frame may be organized differently:

```
inner_join(x = customers, y = orders, by = "id")
```

```
##   id    name order   date
## 1  4   Tukey     1 Jan-01
## 2  8 Wickham     2 Feb-01
## 3 42     Cox     3 Apr-15
```

Question #3: Consider two data frames, "A" and "B", each containing the key variable "ID". In "A", there is a single record for each of the following IDs: $\{1, 2, 3, 4\}$. In "B", there is a single record for each of the following IDs: $\{3, 4, 5, 6\}$. Given this information, state how many rows will be present in the result produced by the following joins:

- **A**: `left_join(x = A, y = B, by = "ID")`
- **B**: `left_join(x = B, y = A, by = "ID")`
- **C**: `full_join(x = A, y = B, by = "ID")`
- **D**: `inner_join(x = A, y = B, by = "ID")`

More on the `by` argument In an ideal world, you have a single key with the same name that can link the information in your "x" and "y" data frames. This is not always the case; however, there are several ways to use the `by` argument to accommodate those scenarios.

- The default value, `by = NULL`, will perform a "natural join", which will automatically identify *all variables* with shared names in "x" and "y" and use that collection to link matching records. This can be useful when several variables are needed to uniquely identify an observation.
- Alternatively, you may use *named vector syntax*, for example: `by = c("a" = "b")`. In this example, the variable named "a" in the "x" data frame will be matched with the variable named "b" in the "y" data frame to link records and perform the join.

Question #4: In the `nycflights13` database, the locations of each airport ("lat" and "lon") are stored in the `airports` data file. For this question, use an appropriate join to add the "lat" and "lon" of the destination airport to each flight recorded in the `flights` data file. *Hint*: the variable "faa" is a primary key in `airports`.

Question #5: The `flights` data frame contains the airport codes for each flight's origin (`origin`) and destination (`dest`). Suppose we want to create a new data frame that includes the full names of both airports for every flight.

- **Part A:** Starting with the original `flights` data frame, use `left_join()` to add the full name of each flight's **origin airport**. Name this new variable `origin_name`.

Hint: You may find it useful to use `select()` and `rename()` on the `airports` data frame before joining.

- **Part B:** Using your result from Part A, perform another `left_join()` to add the full name of each flight's **destination airport**. Name this variable `dest_name`.
- **Part C:** Using your resulting data frame, determine the five most common origin-destination airport pairs. Report the names of both airports and the number of flights for each pair.

Day 2

Joining Multiple Tables

We can also use a series of pipes (`|>`) to join several data frames together without saving intermediate results. When using `|>`, the result of the previous operation is automatically passed as the first argument (`x`) of the next function. This means we only need to specify the second data frame (`y`) and the appropriate keys for each join.

For example, we can add information about both the airline and plane to the `flights` data frame:

```
flights |>
  left_join(airlines, by = "carrier") |>
  left_join(planes, by = "tailnum") |> head(1) # just to check
```

```
## # A tibble: 1 x 28
##   year.x month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##    <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>
## 1   2013     1     1      517            515         2      830            819
## # i 20 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dttm>, name <chr>, year.y <int>,
## #   type <chr>, manufacturer <chr>, model <chr>, engines <int>, seats <int>,
## #   speed <int>, engine <chr>
```

Each join adds information to the result of the previous join. We can continue adding joins or other `dplyr` functions to the same pipeline.

Filtering Joins

Filtering joins will match observations in the same way as mutating joins, but they *will not add any variables* from "y" to the "x" data frame.

- `semi_join()` will **keep** the observations in "x" that have a match in "y"
- `anti_join()` will **drop** the observations in "x" that have a match in "y"

Here are a few examples:

```
semi_join(x = orders, y = customers, by = "id")
```

```
##   order id   date
## 1     1  4 Jan-01
## 2     2  8 Feb-01
## 3     3 42 Apr-15
```

Notice how `semi_join()` filters out the fourth order because there are no customers with an id of 50 in the customers data frame.

```
anti_join(x = orders, y = customers, by = "id")
```

```
##   order id   date
## 1     4 50 Apr-17
```

Conversely, `anti_join()` will filter out everything except for the fourth order because it was the only observation in the "x" data file without a match in the "y" data file.

Filtering joins are particularly useful in finding *implicit missing values*, or values that aren't recorded as `NA` and instead exist only as an absence. For example, we could find missing entries in `airports` by looking for flights whose destination airport doesn't have a match:

```
anti_join(x = flights, y = airports, by = c("dest" = "faa")) |>
  distinct(dest)  ## Return only the unique values of "dest"
```

```
## # A tibble: 4 x 1
##   dest
##   <chr>
## 1 BQN
## 2 SJU
## 3 STT
## 4 PSE
```

We can see that there are four destination airport codes in `flights` without matching records in `airports`.

Question #6: The code below creates a data frame, `top_dest`, containing the ten most common flight destinations and how many flights arrived at them.

```
top_dest = flights |>
  count(dest, sort = TRUE) |>
  head(10)
```

- **Part A:** Use `semi_join()` with `flights` and `top_dest` to create a new data frame containing only flights whose destination is one of the top ten airports. Then, use `dim()` to print the dimensions of the resulting data frame.
- **Part B:** Using your result from Part A, use `anti_join()` with the `planes` data frame to find how many **distinct tail numbers** do not have a matching record in `planes`.

Question #7: The `airlines` data frame contains airline names and their carrier codes, while `flights` contains the carrier code for each flight.

- **Part A:** Use `semi_join()` to find which airlines in `airlines` have at least one flight recorded in `flights` during 2013. Display their names.
- **Part B:** Use `anti_join()` to find which airlines in `airlines`, if any, have no matching flights in `flights`. How many are there?
- **Part C:** Would reversing the roles of `x` and `y` in Part B answer the same question? Explain what the resulting records would represent instead.

Question #8: We can also use filtering joins to investigate which records are missing from a related table. The `planes` data frame contains information about individual planes, identified by `tailnum`.

- **Part A:** Find the distinct tail numbers in `flights` that have no matching record in `planes`. How many distinct tail numbers are there?
- **Part B:** Find the records in `planes` whose tail numbers do not appear in `flights`. How many plane records are there?
- **Part C:** Explain why the results of Parts A and B answer different questions, even though both use the same two data frames and the same key.

Challenge Problem (Required)

Question #9: In Question 5, you used `left_join()` to add the names of the origin and destination airports to the `flights` data frame. You will need to use that same approach as part of this problem.

Without saving intermediate tables, create a data frame containing exactly six columns: the plane model, airline name, hour, temperature, origin airport name, and destination airport name for all flights on October 9, 2013.

Your solution should use a series of pipes (`|>`), joining the necessary tables and selecting the requested variables. Do not save intermediate tables in your final solution.

Hints: Your resulting data frame should have **974 rows and 6 columns**. You will need to use information from several tables in the `nycflights13` database, and some tables may need to be joined more than once.

```
## # A tibble: 5 x 6
##   model    airline     timeOfDepart temperature originAirport destinationAirport
##   <chr>    <chr>              <dbl>       <dbl> <chr>         <chr>
## 1 A321-231 US Airways~            5        51.1 Newark Liber~ Charlotte Douglas~
## 2 757-222  United Air~            5        51.1 Newark Liber~ George Bush Inter~
## 3 737-724  United Air~            5        55.9 La Guardia    George Bush Inter~
## 4 A320-232 JetBlue Ai~            5        53.1 John F Kenne~ <NA>
## 5 <NA>     American A~            5        53.1 John F Kenne~ Miami Intl
```